Day 108 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 4 reference

Watch: 13.4.2 Feature Permutation Importance (Sebastian Raschka) | 5.5 Permutation Feature Importance (KHU IAILab)

`day108_feature_importance.py`, segment by segment, in the order `main()` uses things. The forest is Day 107's, imported unchanged; scikit-learn is used only as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: MDI = mean decrease in impurity · PI = permutation importance · n_{node} = rows reaching a question · r = correlation

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day107_random_forests as d107
d106, d105 = d107.d106, d107.d105
fit_forest, forest_proba, accuracy = d107.fit_forest, d107.forest_proba, d107.accuracy
stratified_split = d105.stratified_split
TREES = 100
REPEATS = 10
SPLITS = 10
KEEP = [1, 2, 4, 8, 16, 32, 64]
```

- `d106, d105 = d107.d106, d107.d105` — the earlier lessons, through the modules Day 107 already loaded.
- `REPEATS = 10` — shuffles per feature; more shuffles reduce the noise but cost one forest prediction each.
- `KEEP = [1, 2, 4, 8, 16, 32, 64]` — doubling, because the interesting changes happen at small counts.

Segment 1 — The measures

tree_importance and impurity_importance

MATH

$$MDI(f) = \Sigma_{\text{questions using } f} (n_{node} / n) \times \text{gain, then shares, then averaged over trees}$$

CODE

```
def tree_importance(tree, p):
    out = np.zeros(p); stack = [tree]; n = tree["counts"].sum()
    while stack:
        node = stack.pop()
        if "feature" in node:
            out[node["feature"]] += node["counts"].sum() / n * node["gain"]
            stack += [node["left"], node["right"]]
    return out / max(out.sum(), 1e-12)
def impurity_importance(trees, p):
    return np.mean([tree_importance(t, p) for t in trees], axis=0)
```

- `stack += [node["left"], node["right"]]` — adds both children to the to-do list: a walk with no recursion.
- `node["counts"].sum() / n * node["gain"]` — weighted by the share of rows reaching this question.
- `out / max(out.sum(), 1e-12)` — shares summing to 1; max protects a one-leaf tree from dividing by zero.

permutation_importance

MATH

$$PI(f) = score(X) - \text{average of } score(X \text{ with } f \text{ shuffled})$$

CODE

```
def permutation_importance(trees, X, y, k, seed, groups=None):
    rng = np.random.RandomState(seed); base = accuracy(forest_proba(trees, X, k), y)
    groups = groups if groups is not None else [[j] for j in range(X.shape[1])]
    out = np.zeros((len(groups), REPEATS))
    for g, cols in enumerate(groups):
        for r in range(REPEATS):
            Z = X.copy(); Z[:, cols] = X[rng.permutation(len(X))][:, cols]
            out[g, r] = base - accuracy(forest_proba(trees, Z, k), y)
    return out.mean(1), out.std(1)
```

- `groups if groups is not None else [[j] for j in ...]` — a conditional expression: by default each feature is its own group.
- `X[rng.permutation(len(X))][:, cols]` — the rows in shuffled order, then only these columns.
- `Z[:, cols] = ...` — assigns several columns at once, so a group moves together and stays consistent.

spearman

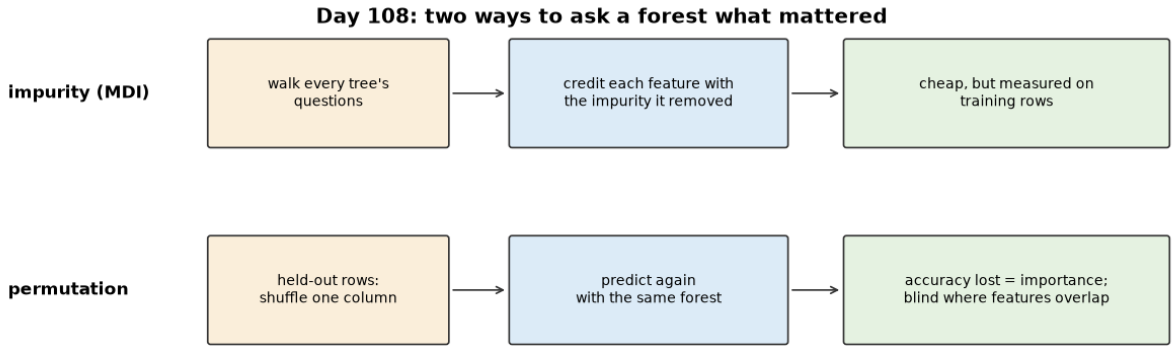
CODE

```
def ranks(a):
    r = np.empty(len(a)); r[np.argsort(a, kind="stable")] = np.arange(1, len(a) + 1)
    return r
def spearman(a, b):
    return float(np.corrcoef(ranks(a), ranks(b))[0, 1])
```

- `r[np.argsort(a, kind="stable")] = np.arange(1, len(a) + 1)` — writes rank 1 at the smallest value's position, 2 at the next, and so on.

- `np.corrcoef(u, v)[0, 1]` — a 2 by 2 correlation matrix; the off-diagonal is the correlation.

SYSTEM DIAGRAM



Two ways to ask a forest what mattered.

Segment 2 — The experiments

clusters: groups of near-copies

CODE

```
def clusters(X, limit=0.9):
    r = np.abs(np.corrcoef(X.T)); left = list(range(X.shape[1])); out = []
    while left:
        j = left.pop(0); g = [j] + [i for i in left if r[j, i] > limit]
        left = [i for i in left if i not in g]; out.append(g)
    return out
```

- `np.corrcoef(X.T)` — corrcoef treats rows as variables, so the table is transposed.
- `j = left.pop(0)` — takes the first feature not yet placed.
- `[i for i in left if r[j, i] > limit]` — its near-copies; greedy, so the order of the features matters a little.

titanic_extended: two columns that cannot matter

CODE

```
def titanic_extended():
    X, y = d106.load_titanic()
    pid = pd.read_csv(d105.TITANIC)["PassengerId"].to_numpy(float)
    coin = np.random.RandomState(0).randint(0, 2, len(y)).astype(float)
    return np.column_stack([X, pid, coin]), y, d106.FEATS + ["PassengerId", "coin"]
```

- `np.column_stack([X, pid, coin])` — puts one-dimensional arrays beside a table as new columns.
- `rng.randint(0, 2, len(y))` — a fair coin per passenger: 0 or 1.

Drop-column, inside demo_useless

CODE

```
for j in range(p):
    keep = [i for i in range(p) if i != j]
    t2, _ = fit_forest(X[tr][:, keep], y[tr], 2, TREES, 3, s)
    row.append(base - accuracy(forest_proba(t2, X[te][:, keep], 2), y[te]))
```

- `keep = [i for i in range(p) if i != j]` — every column except one.

- `X[tr][:, keep]` — rows first, then columns: two steps, because indexing both at once would pair the lists up.
- `base - accuracy(...)` — positive means the forest was better with the feature.

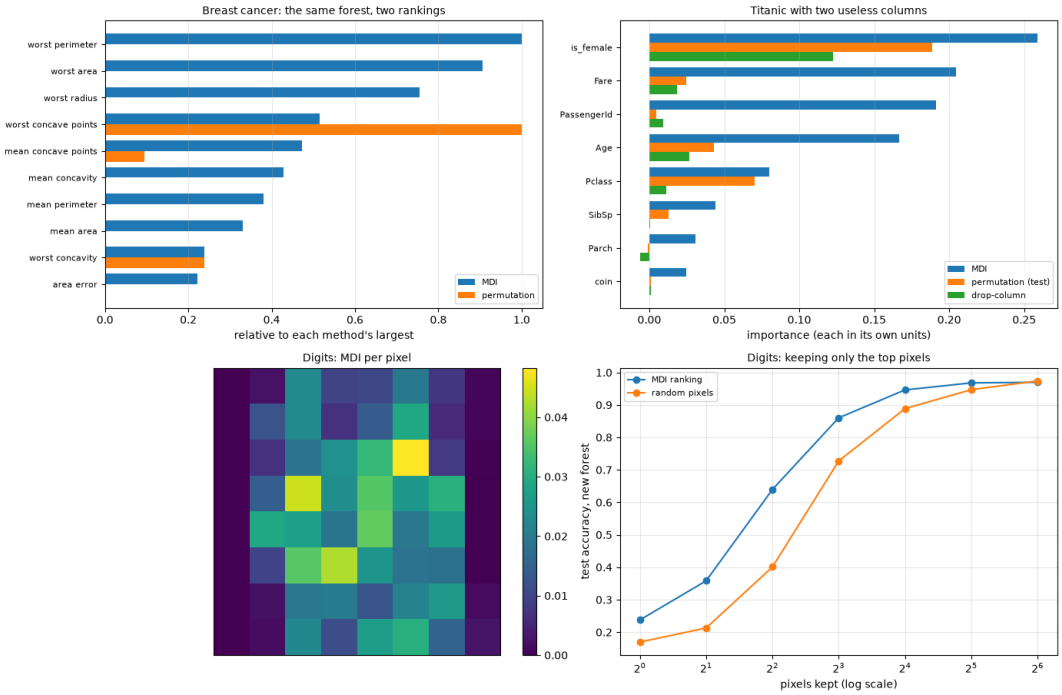
Choosing pixels, inside demo_digits

CODE

```
c = o[:k]; f, _ = fit_forest(X[tr][:, c], y[tr], 10, TREES // 2, d107.sqrt_features(k), s)
r.append(accuracy(forest_proba(f, X[te][:, c], 10), y[te]))
```

- `c = o[:k]` — the first k columns of a ranking made on training rows only.
- `d107.sqrt_features(k)` — mtry follows the number of columns kept.

REAL GRAPH



Real graph from the script: two rankings, Titanic with useless columns, and digits.

Quick Syntax Glossary — new or reinforced constructs

- `a if cond else b` — a conditional expression.
- `X[perm][:, cols]` — shuffle the rows, then choose columns.
- `Z[:, cols] = values` — assign several columns at once.
- `np.column_stack(list)` — join arrays side by side.
- `np.corrcoef(X.T)` — correlations between the columns of X.
- `map(np.array, (a, b))` — apply a function to several items, for unpacking.
- `list.pop(0)` — remove and return the first item.

End of Day 108 syntax reference. For the experiments and what they showed, see the Day 108 complete guide.